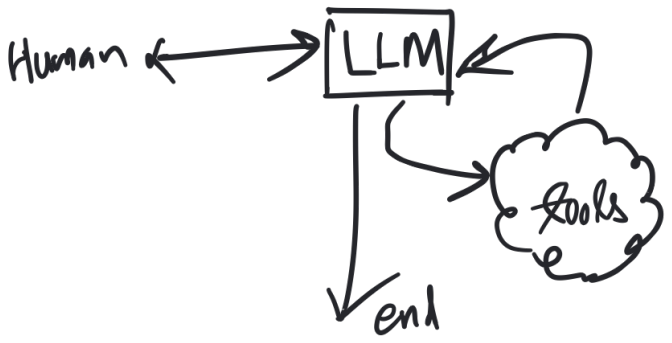



OpenAI
HuggingFace
Ollama
Anthropic ...

**LLMs**

generative

predict the next token
**(create content)**

reasoning

implies a kind of cognition
**(reason & react to tools)**

tool calling

**An agent is born**

Role Playing worker

**Agent**

- Agents perform better when role-playing
  You are a Researcher, your job is to collect reliable info
- Focus an agent on a goal & expectation

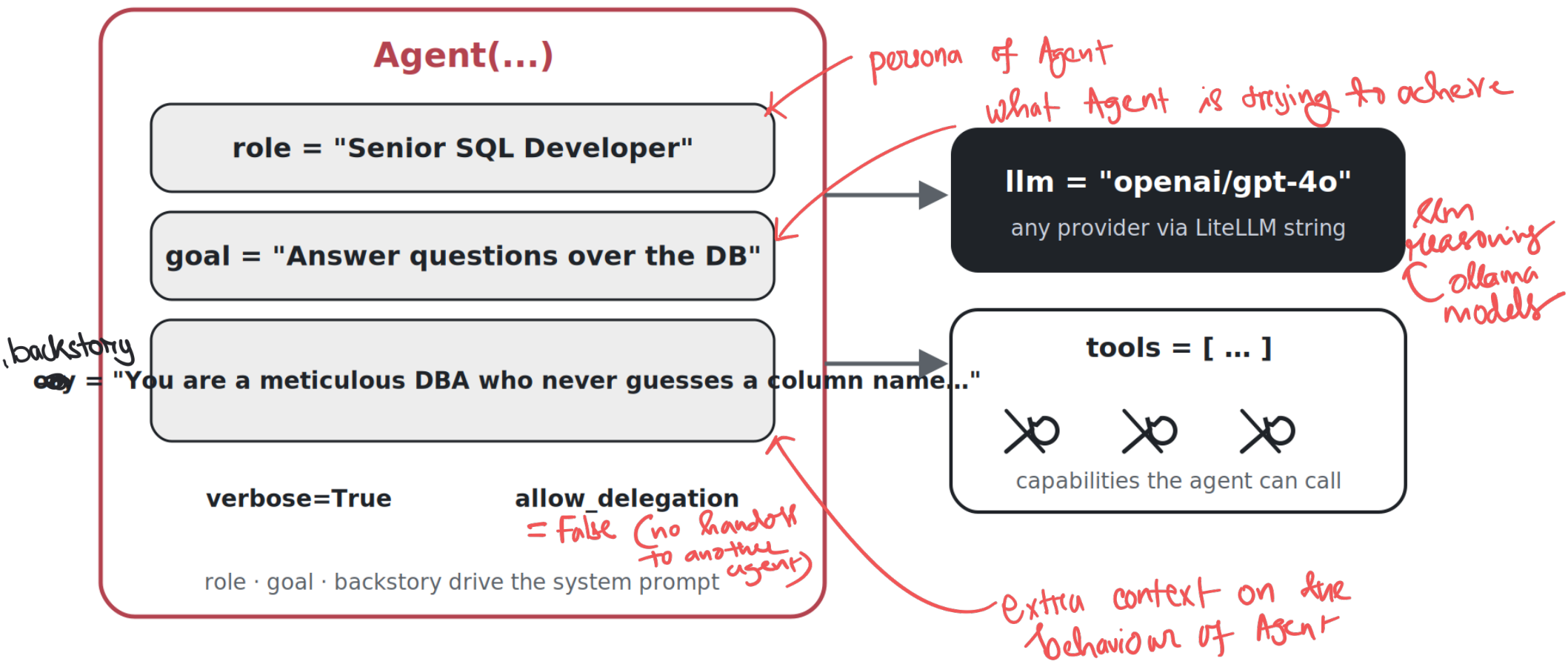
**Task**

Unit of work assigned to agent

- One agent can own multiple tasks

**Crew**

- Keep tasks & agents granular
  Own the agents and tasks (run them in a process)
- A Crew runs the tasks (sequential / hierarchical)


Agent(...)
role = "Senior SQL Developer"
goal = "Answer questions over the DB"
backstory = "You are a meticulous DBA who never guesses a column name..."
verbose=True
allow_delegation
= False (no handoff to another agent)
role · goal · backstory drive the system prompt
persona of Agent
what Agent is trying to achieve
llm = "openai/gpt-4o"
any provider via LiteLLM string
llm reasoning (ollama models)
tools = [ ... ]
capabilities the agent can call
extra context on the behaviour of Agent

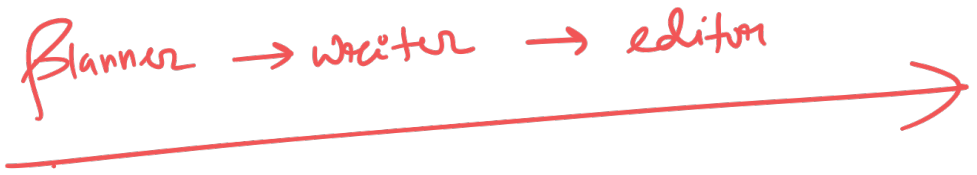
Task options used across L5/L7:

- `output_json=MyPydanticModel` — force a typed, structured result.
- `output_file="report.md"` — also persist the result to disk.
- `context=[task_a, task_b]` — wait for those tasks and receive their outputs.
- `async_execution=True` — overlap with later tasks.
- `human_input=True` — pause for human approval before finalising.
- `tools=[...]` — task-level tools (see *Tools* below).

```python
from pydantic import BaseModel

class VenueDetails(BaseModel):
    name: str; address: str; capacity: int; booking_status: str

venue_task = Task(
    description="Find a venue in {city} that fits {topic}.",
    expected_output="All details of the chosen venue.",
    agent=venue_coordinator,
    output_json=VenueDetails, output_file="venue.json", human_input=True,
)
```




## Crew & `kickoff`

The Crew binds agents to tasks; `kickoff(inputs=...)` interpolates every `{placeholder}`.

```python
from crewai import Crew

crew = Crew(agents=[planner, writer, editor],
            tasks=[plan, write, edit], verbose=True)   # sequential by default
result = crew.kickoff(inputs={"topic": "Artificial Intelligence"})
```

**Prebuilt tools** (`crewai_tools`) — used in L4/L5/L6/L7:

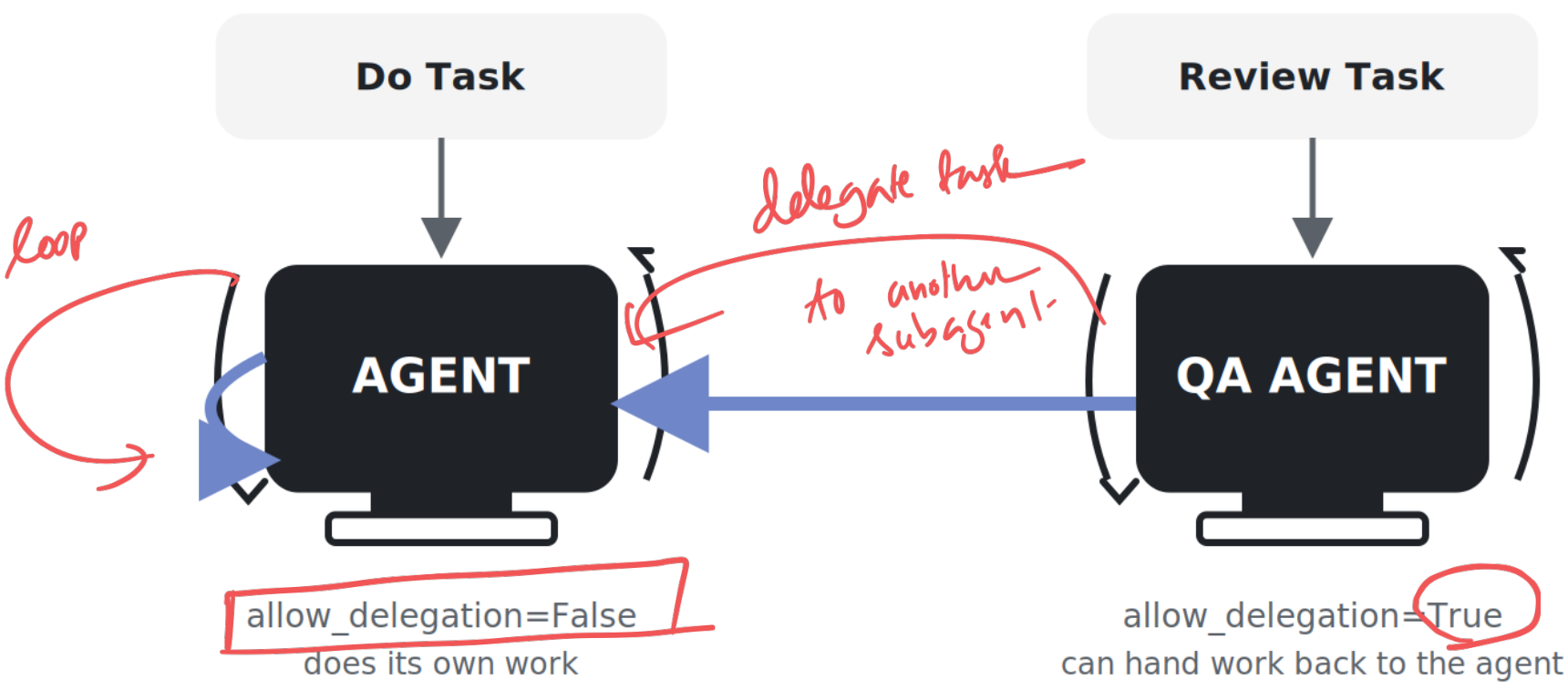
```
from crewai_tools import (SerperDevTool, ScrapeWebsiteTool,
                          DirectoryReadTool, FileReadTool, MDXSearchTool)
search = SerperDevTool()                                    # web search (SERPER key)
scrape = ScrapeWebsiteTool(website_url="https://docs.crewai.com/")
rag    = MDXSearchTool(mdx="./resume.md")                   # semantic search over a file
```

**Custom tool — way 1:** `@tool` (quick, function-style):

```
from crewai.tools import tool

@tool("Sentiment Analysis Tool")
def sentiment(text: str) -> str:
    "Analyze sentiment to keep communication positive."
    return "positive"
```

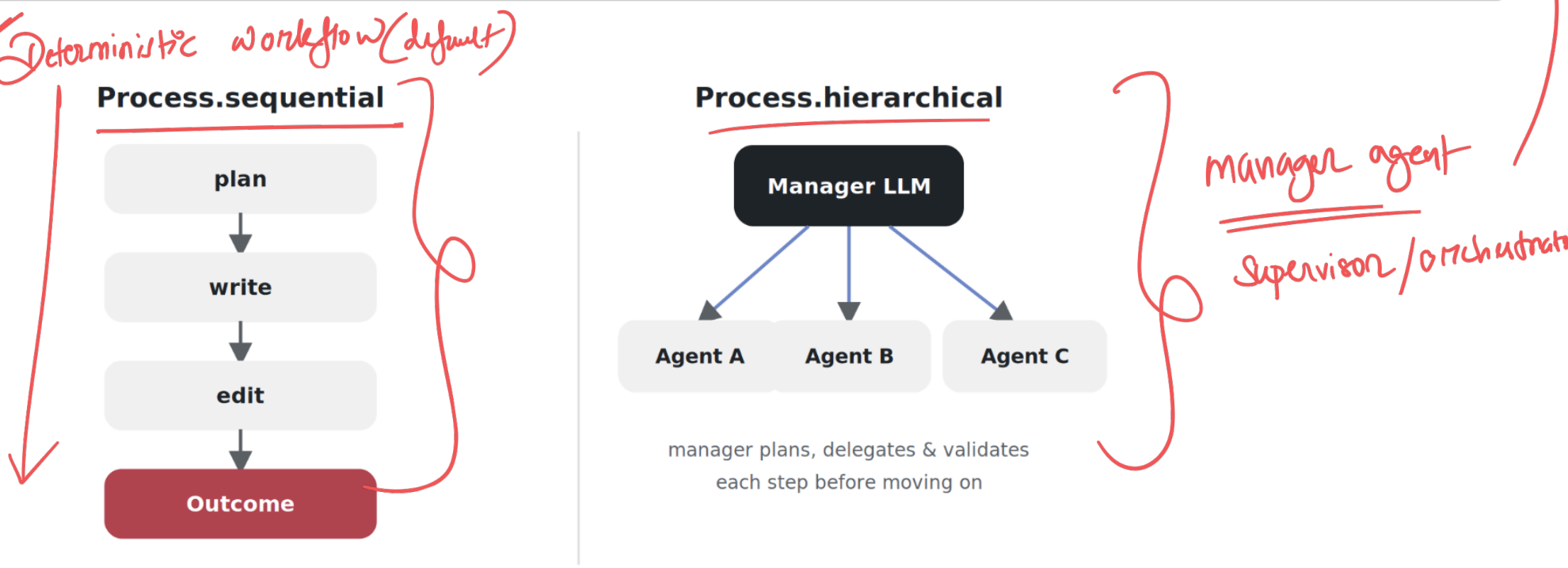
# allow_delegation



## Processes: sequential vs hierarchical

**Sequential** (default) runs tasks in list order. **Hierarchical** (L6) adds a **manager LLM** that plans, delegates, and validates each step.

```
from crewai import Crew, Process
crew = Crew(agents=[...], tasks=[...], process=Process.hierarchical,
            manager_llm=LLM(model="openai/gpt-4o", temperature=0.7))
```



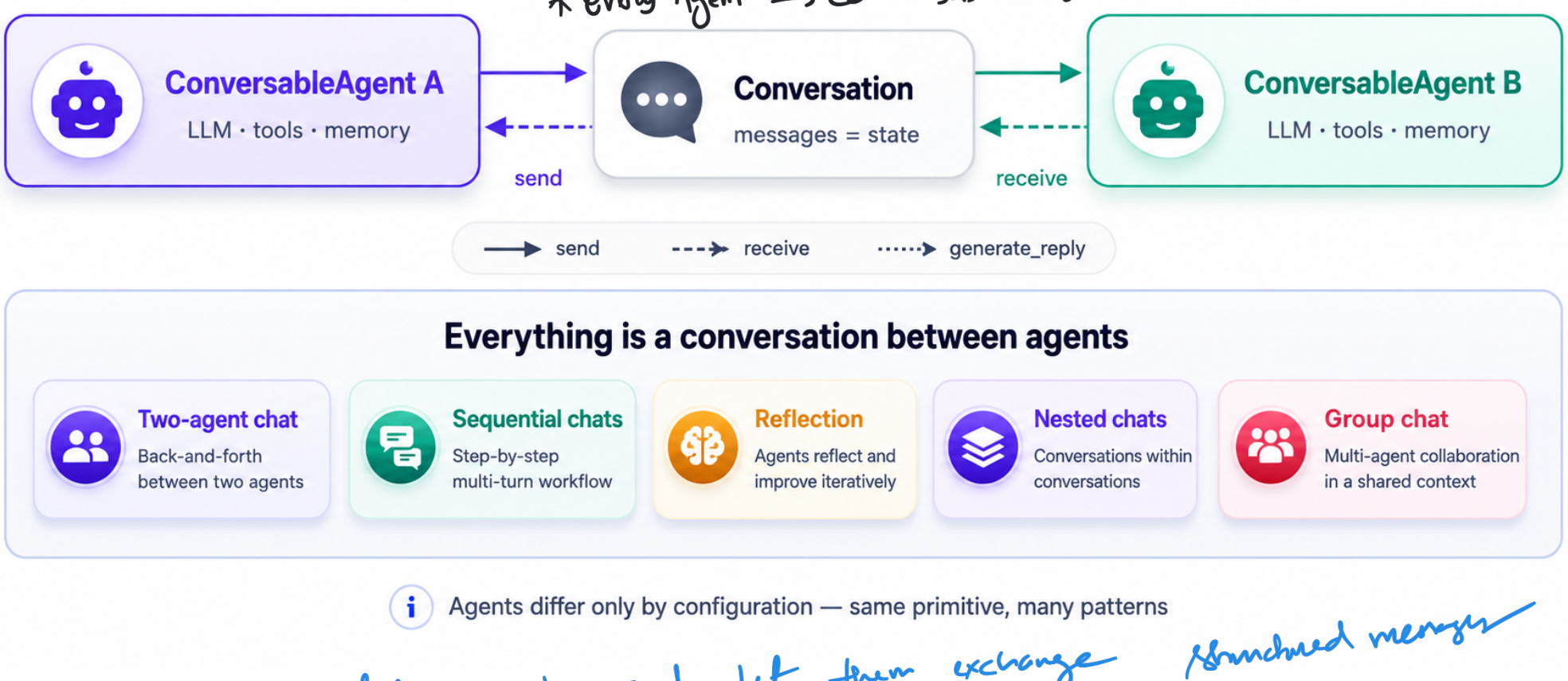
10 mins

agentic app => conversat^n bet^n agents

* no manual control flow

# AutoGen — Multi-Agent Conversation Framework

* Every Agent → Conversable Agent




## Everything is a conversation between agents

**Two-agent chat**
Back-and-forth between two agents

**Sequential chats**
Step-by-step multi-turn workflow

**Reflection**
Agents reflect and improve iteratively

**Nested chats**
Conversations within conversations

**Group chat**
Multi-agent collaboration in a shared context

Agents differ only by configuration — same primitive, many patterns

* define agents and let them exchange structured messages

langgraph

invoke

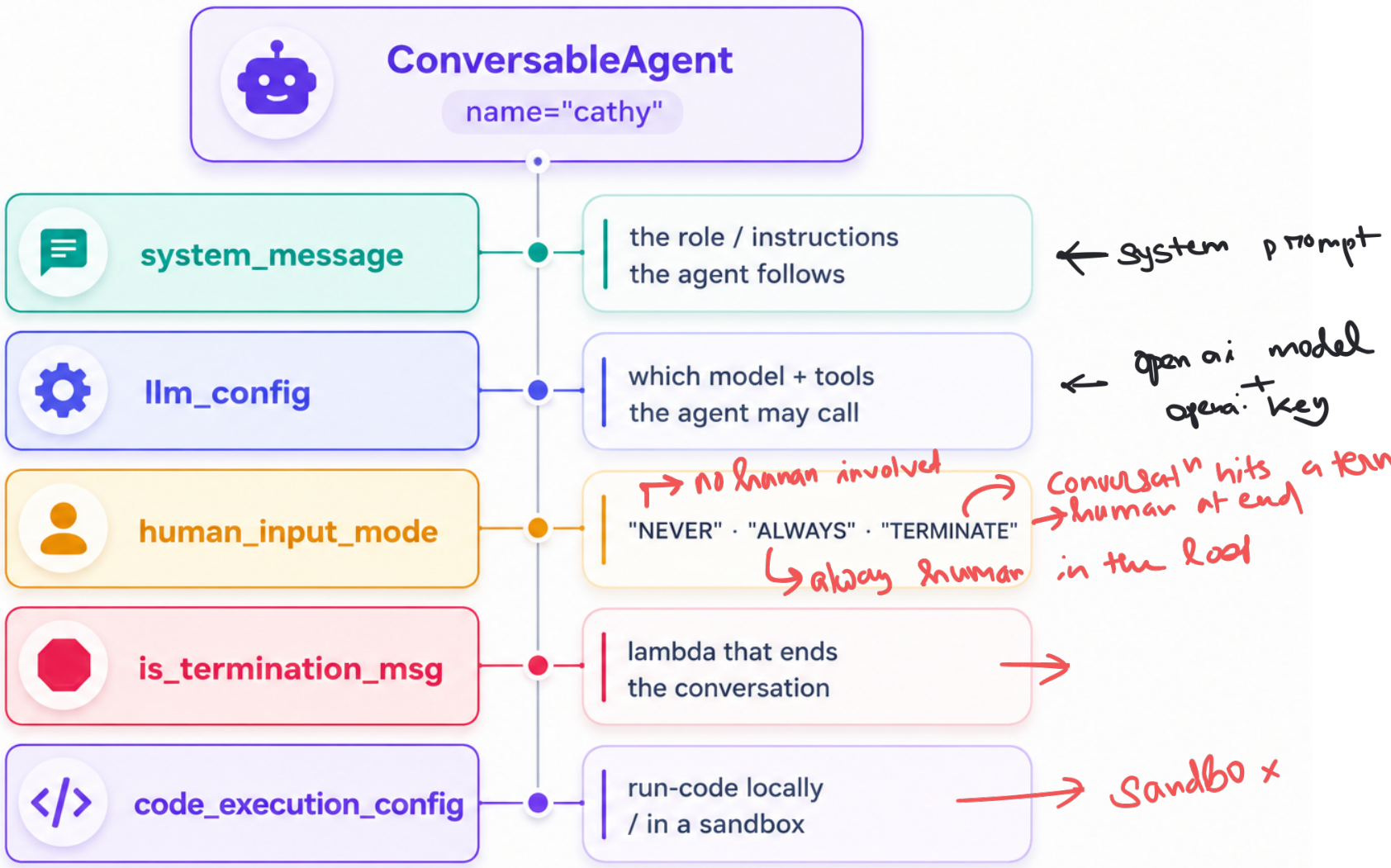
autogen

initiate_chat

crew.ai

kickoff

# Anatomy of a ConversableAgent

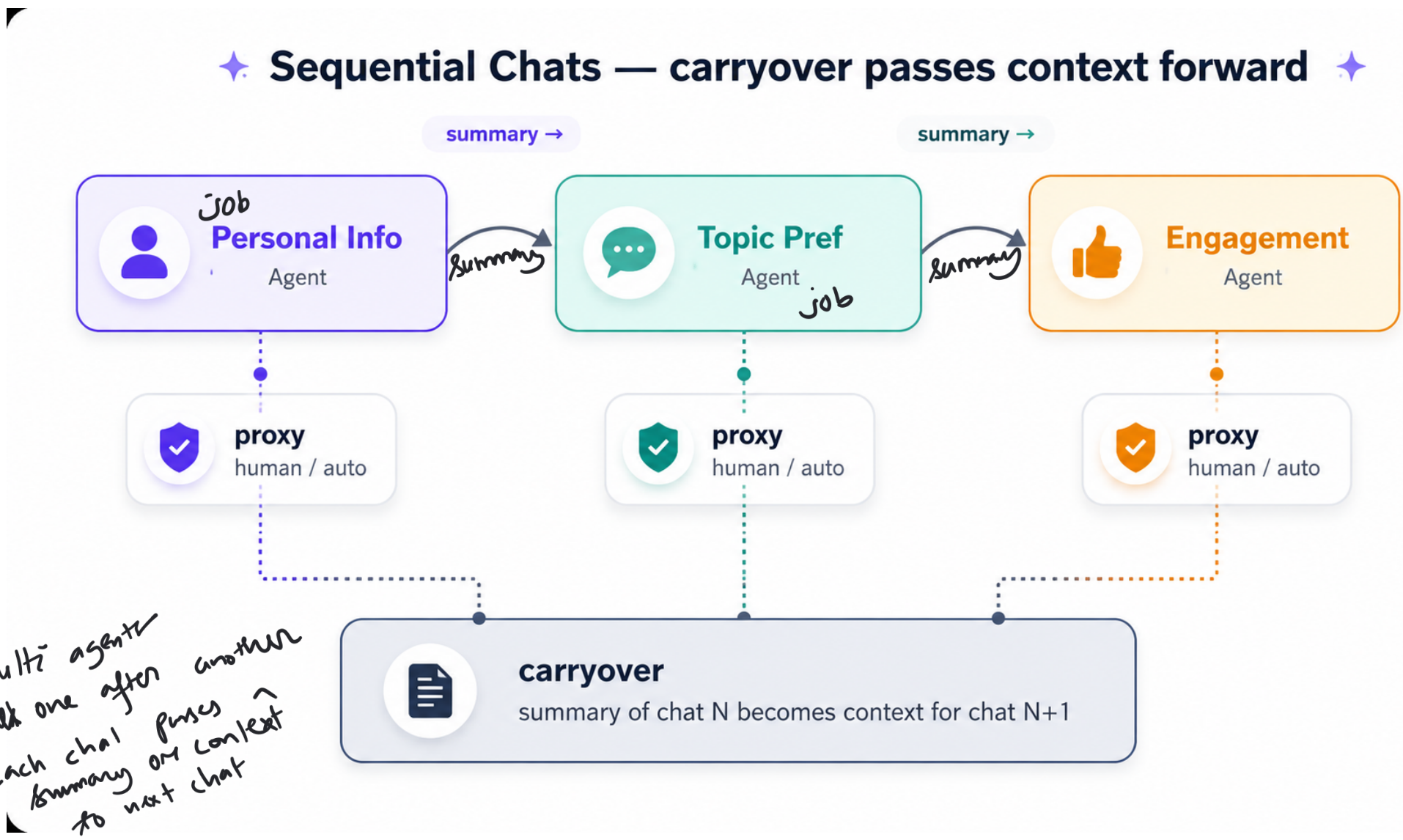


Cathy

— Becz they make up everything

Joe—

Speaking of making things up — — —


Sequential Chats — carryover passes context forward
summary →
summary →
Personal Info
Agent
Topic Pref
Agent
Engagement
Agent
proxy
human / auto
proxy
human / auto
proxy
human / auto
carryover
summary of chat N becomes context for chat N+1